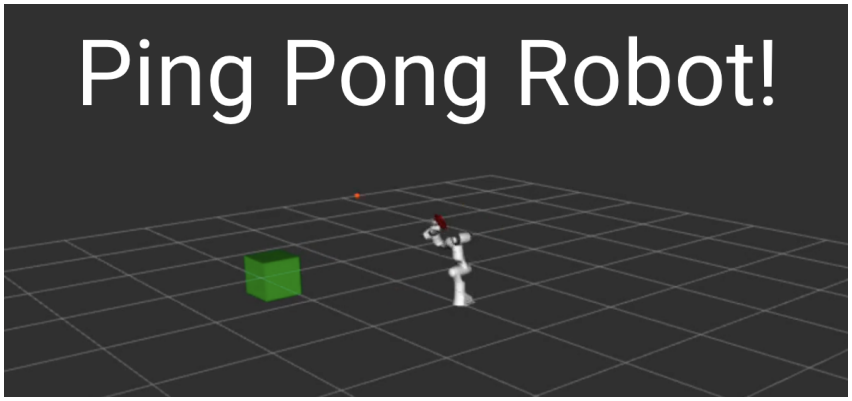
ME/CS/EE 133a Final Report

Ashiria Goel, Emily Xu, Ana Jaramillo

December 12, 2025



Introduction and Objectives

This project focuses on utilizing kinematics and analysis of degrees of freedom to smoothly move a robot. Using the Franka Panda robotic arm, this project aims to move a paddle attached to the end-effector to intercept a falling ball and hit it towards a target goal. The system uses physics and dynamics to predict the time a ball will take to reach a certain intercept height for the robot to hit it at. Based on this predicted intercept, the robot plans a smooth motion to reach the ball and orients the paddle so that the outgoing ball trajectory is directed towards the goal. This task introduces additional requirements as the ball is subject to gravity, the interception between the paddle and the ball involves physical contact and a collision response, and both the ball spawn location and the goal position are randomly assigned per iteration. Due to these project parameters, the robot must re-plan its motion every time.

System Description

a) The Robot

The robot used in the project is a Franka Panda equipped with a paddle at the end effector such that there is a fixed joint between the wrist and the paddle. The Franka Panda originally has seven revolute degrees of freedom, which provide kinematic redundancy for task-space control. To increase flexibility in where the ball can be intercepted, two additional prismatic degrees of freedom (creating a total of 9 degrees of freedom) were added at the paddle, allowing translational motion of the tip in the plane of the paddle.

The workspace of the robot corresponds to the reachable region, which from the top view spans a full circle. However, the side view is limited by the arm's link lengths and joint constraints. The robot is fixed to the ground and the kinematic redundancy allows for multiple joint configurations to perform the same task.

To integrate the paddle onto the robot, modifications were made to the robot's URDF. and STL for a ping-pong paddle as well as two prismatic joints and their associated link. Because the geometric center of the STL did not coincide with the physical center of the paddle surface, the origin of the paddle link was offset in the URDF to correctly align the collision surface with the robot's kinematics.

b) Additional Elements

In addition to the robot, the system includes a ball and goal marker which together help make a dynamic task environment.

The ball's motion is simulated using simple physics. Each time step, the ball's vertical velocity is updated based on gravity and the position is then integrated forward in time. Once the ball comes into contact with the paddle or the floor, its velocity changes using a restitution-based collision model. Once the motion of the ball becomes negligible, the ball is respawned.

When the ball contacts the paddle, the velocity s updating using a momentum-based collision model. The ball's velocity is decomposed into components normal and tangential through the paddle surface. The tangential component is preserved through the collision, while the normal component is updated according to conservation of momentum between the ball and moving paddle. Let $\mathbf{n}$ denote the unit normal of the paddle surface in world coordinates, $\mathbf{v}_b$ be the ball velocity and $\mathbf{v}_p$ be the paddle velocity, and m_b, m_p be the masses of the ball and the paddle. The post-impact normal velocity of the ball is determined using a one-dimensional collision model along $\mathbf{n}$. This allows the outgoing ball velocity to depend on the paddle motion at impact.

Along $\mathbf{n}$ we apply conservation of linear momentum and a coefficient of restitution e on the relative normal velocity (ball to paddle). We use the following equations,

$$v_{b,n} = \mathbf{v}_b \cdot \mathbf{n}, \qquad v_{p,n} = \mathbf{v}_p \cdot \mathbf{n}.$$

The tangential component of the ball velocity is defined as

$$\mathbf{v}_{b,t} = \mathbf{v}_b - v_{b,n}\mathbf{n}.$$

Along the normal direction, conservation of momentum and the restitution condition (using the definition of coefficient of restitution as relative velocity after collision divided by relative velocity before collision) are given by

$$m_b v_{b,n}^- + m_p v_{p,n}^- = m_b v_{b,n}^+ + m_p v_{p,n}^+, \tag{1}$$

$$v_{b,n}^+ - v_{p,n}^+ = -e\left(v_{b,n}^- - v_{p,n}^-\right), \tag{2}$$

The $(-)$ and $(+)$ are pre and post impact velocities, respectively.

When solving for the required paddle velocity, we use the collision model to compute the paddle's required normal velocity to achieve a desired ball launch velocity (solved from the projectile motion equation).

Using the desired $v_{\text{launch},n}$ along $\mathbf{n}$ and solving for $v_{p,n}^{-}$ gives

$$v_{p,n}^{-} = \frac{(m_b + m_p)\, v_{\text{launch},n} - (m_b - e m_p)\, v_{b,n}^{-}}{(1+e)m_p}. \tag{3}$$

First solved for equation 4, then reversed it using v_b^+ as the calculated launch velocity, to get equation 3. The derivation is in the bottom of this report.

We also set the mass of the ball to be 0.02 and the mass of the paddle to be 2, so the velocity of the paddle before and after collision has very little difference. Because the paddle is much heavier than the ball, we are able to approximate $v_p^+ = v_p^-$ (this can also be proved using the equations, but I won't derive it here).

When the ball and paddle collide, we use the same equation to solve for the post-impact ball velocity. Because we are using the same equations for solving for paddle velocity (from desired ball velocity), then using paddle velocity to solve for ball velocity, we get that the post-impact ball velocity is designed to match the desired launch velocity. In practice, because of our numerical integration, it matches up very closely.

During planning, we invert the 1D collision model to compute the required paddle normal velocity that produces the desired launch velocity, during execution we apply the forward equation to compute

$$v_{b,n}^{+} = \frac{(m_b - e m_p)\, v_{b,n}^{-} + ((1+e)m_p)\, v_{p,n}^{-}}{m_b + m_p} \tag{4}$$

The full post-impact ball velocity is then reconstructed by combining the unchanged tangential component with the updated normal component:

$$\mathbf{v}_b^{+} = \mathbf{v}_{b,t} + v_{b,n}^{+}\, \mathbf{n}. \tag{5}$$

This formulation allows the outgoing ball velocity to depend on the paddle velocity at the moment of impact.

The goal marker represents a static target location that the ball is expected to reach after being struck by the robot. At each iteration, the goal position is randomly generated. The position is then published both as a visualization marker which can be seen in Figure 1 as the green cube and as a point message that the robot controller uses to determine the desired outgoing ball trajectory. This node is also aware of the ball's position by subscribing to the ball position topic. This is done so that if the distance between the ball and the goal falls below a specified threshold, the goal is marked as reached. Once reached the goal respawns to a different location for the task to be performed again.

c) Tasks

The objective of this project is to intercept and hit a ball such that it reaches a specified goal location. This objective leads to defining a task that places the paddle at a planned intercept point (x, y, z) with an orientation that aligns the paddle normal with the desired direction of the ball.

The primary task is a six-dimensional task where there are 3 translational dimensions and 3 rotational dimensions. This then results in a six-dimensional error vector combining both the position and orientation errors. The task is defined by specifying both the desired position and a desired orientation for the paddle. The desired position describes where

the center of the paddle should be located in the three-dimensional space relative to the robot base using cartesian coordinates. This essentially determines where the paddle needs to be at the moment that it needs to make contact with the ball. The desired orientation, on the other hand, describes the rotation of the paddle to ensure the ball reflects off in the direction of the goal. At each iteration, the current paddle pose (p_c, R_c) is computed using forward kinematics and the task-space error is computed comparing the current pose to the desired pose. The task coordinates are $\mathbf{x} = \begin{bmatrix} \mathbf{p} \\ \mathbf{R} \end{bmatrix}$ where p = [x, y, z] is the paddle position and R is the paddle orientation.

d) Task Organization

The tasks are structured as a primary task then a secondary task. The primary task is to place the paddle at the desired position and orientation at the correct time so that the valid collision happens with the ball. This task must always be satisfied. As a secondary task, the two prismatic joints are regulated. These are biased towards a zero displacement, encouraging impacts near the center of the paddle.

The current system coordinates multiple subsystems including the physics of the ball and the trajectory prediction which determine when and where an interception must occur. There is also the task-space planning, which commutes the desired paddle pose and velocity. The inverse kinematics maps the task objectives to joint commands, and the splines execute the motion smoothly.

The task is not always achievable due to workspace limits, joint limits, and timing constraints. Cases in which the task is not achievable is handled in the implementation by stating that if the inverse kinematics fails to converge or reaches a minimum error state, the planner detects this condition and avoids executing an invalid intercept. Additionally, intercept planning only occurs when the ball is above the paddle and moving downwards. This is done to help avoid unrealistic plans.

Conflicts are handled through the nullspace projection. The secondary task is projected into the nullspace of the Jacobian such that it does not interfere with the primary task. This helps guarantee that the secondary task is only done when the accuracy of the primary task is not at stake.

Algorithm, Implementation, and Mathematical Details

a) Trajectories and Driving Elements

The system runs on multiple iterations. With each iteration, it updates the ball state and also checks whether a new interception plan is needed which will then move the paddle at the correct time. The ball is updated using the following simple kinematic equations under a gravity constant:

$$\dot{z} = v_z$$

$$\dot{v}_z = g$$

, $g < 0$ When the ball is above the paddle and moving downward, a new intercept is planned and it is computed by solving for when the ball reaches the paddle height (z_{hit}).

$$z(t) = z_0 + v_{z0}t + \frac{1}{2}gt^2$$

$$\frac{1}{2}gt^2 + v_{z0}t + (z_0 - z_{\text{hit}}) = 0$$

$$t_{\text{hit}} = \frac{-v_{z0} \pm \sqrt{v_{z0}^2 - 2g(z_0 - z_{\text{hit}})}}{g}$$

and select the positive root.

The target paddle position is then chosen to match the ball's predicted horizontal position at the intercept time. In addition to planning the paddle position at intercept time t_{hit} the ball's velocity immediately before impact is also predicted. We predict the incoming ball velocity at impact by propagating the current velocity forward to t_{hit} under gravity (only affecting the z component). This predicted velocity is used, with a conservation of momentum in a collision, to calculate the desired outgoing velocity and trajectory. The paddle outgoing velocity is also used to specify the normal to the paddle, which is used to calculate a rotation matrix for the desired orientation of the paddle.

The orientation of the paddle is also taken into account. The paddle normal is chosen to align with the change needed between incoming and outgoing velocities. The n vector is determined by subtracting the ball velocity at impact from the ball velocity at launch and the rotation matrix is constructed so that the paddle's local positive z axis is the n vector. We only constrain the surface normal. We compute an orthonormal basis around this normal by selecting a vector not colinear with the normal, computing a perpendicular axis via a cross product, then computing the third axis as the cross product of the normal with the first perpendicular axis. These three mutually orthogonal unit vectors are then used as the columns in the rotation matrix. This approach ensures a valid paddle orientation while preserving rotational freedom within the plane of the paddle.

b) Inverse Kinematics

The inverse kinematics is solved from the paddle pose. At each iteration of the inverse kinematics, the current pose is computed using forward kinematics forming a six dimensional task error consisting of position and orientation components.

$$\mathbf{e} = \begin{bmatrix} \mathbf{e}_p \\ \mathbf{e}_R \end{bmatrix} = \begin{bmatrix} \mathbf{p}_d - \mathbf{p}_c \\ \mathbf{e}_R(R_d, R_c) \end{bmatrix}$$

The primary objective of this project is to place the paddle at the desired position and orientation to be able to collide with the ball. A Jacobian matrix is used to describe how small changes in the robot's joint angles affect changes in the paddle's position and orientation. Joint angles that move the paddle in the correct direction in task space are computed. Because the robot has more joints than task dimensions, the Jacobian cannot be inverted directly, thus a damped pseudoinverse is used, which produces a joint update that best reduces the task error.

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\mathbf{p}} \\ \boldsymbol{\omega} \end{bmatrix} = \begin{bmatrix} J_v \\ J_\omega \end{bmatrix} \dot{\mathbf{q}}$$

More specific to this implementation, a weighted damped pseudoinverse is used to stay clear of more expensive joints.

$$J_d = W^{-1}J^\top \left(JW^{-1}J^\top + \gamma^2 I\right)^{-1}$$

This helped discourage the motion of certain joints—in this case, joint 4.

We solve inverse kinematics using an iterative Newton-Raphson iterative method on the nonlinear pose constraint e = 0. Because the Jacobian is non-square, the joint upate is computed using a damped least-squares approximation of the Newton step. We have that

$$\Delta q_{primary} = cJ^{-1}(q_{i-1})\mathbf{e}(x, fkin(q_{i-1}))$$

with step size c. Then, we can slowly converge to a joint solution $q_i = q_{i-1} + \Delta q$More information about how we selected step size c is in the analysis section of the report.

In defining the orientation error for the paddle, we deliberately constrain only the direction of the paddle normal and leave rotations within the paddle unconstrained. Although the full orientation error is computed (using the rotation error function), we express this error in the paddle's local frame and zero out the component corresponding to rotation about the paddle normal. This reflects the fact that the task only requires the paddle surface to face the correct direction (so the ball reflects to the goal, rotation about the normal doesn't matter). We then map this from the paddle frame into the world frame, the IK solver is prevented from unnecessarily twisting the wrist to satisfy the irrelevant rotational constraint. This reduces joint motion and improves convergence, allowing the robot to explicitly define forehand or backhand and guaranteeing the correct normal.

Because the inverse kinematics is solved iteratively, we explicitly classify convergence outcomes using both the task-space error norm and step size norm. Let $\mathbf{e}$ be the combined position-orientation error magnitude nad Δq be the joint update magnitude. We define a successful convergence ("reached") when the solver achieves a small task error and the update step is also small. We define a "stuck" condition when the update magnitude becomes negligible but the residual task-space error is above the bound, corresponding to unreachable targets. Finally, if neither condition occurs within our fixed iteration of 500 steps, we label the solve as "oscillating," indicating that iterates don't settle to a specific solution. This allows the planner to reject invalid intercepts rather than spasming out.

```
    [within iteration steps]
        if err_norm < tol and dq_norm < dq_tol:
        return q, "reached"

    # Case 2: Stuck at local minimum (step size small but error still large)
    if dq_norm < dq_tol and err_norm > tol:
        return q, "stuck"

    q = q + dq
return q, "oscillating"
```

The weighting matrix (W) introduces a tradeoff that minimizes the weighted joint motion norm while still achieving the task-space objective.

Concatenation of actions happens once q_{goal} and $\dot{q}_{\text{goal}}$ are computed and it generates a motion using two joint-space cubic splines. One that splines from q_0 to q_{goal} at the intercept time t_{hit} and the other that splines from q_{goal} to q_0 over a return time t_{return}. The second spline was implemented to help show a more natural swing-like motion of hitting a ball–similar to how tennis players swing through after hitting the ball.

In addition to desired paddle pose, a desired paddle linear velocity is specified at the moment of impact. This velocity is chosen such that, under the collision model, the ball leaves teh paddle with the desired outgoing velocity. The desired paddle velocity v_{paddle} is mapped to joint-space velocities using the translational Jacobial, $\dot{q}_{goal} = J_v^t v_{paddle}$, where J_v^t is the damped pseudoinverse of the translational jacobian. This is done to avoid large joint velocities near singular configurations. The resulting joint velocity is used as the final velocity for the forward spline segment, and the initial velocity for the return spline segment. This further ensures the paddle arrives at the intercept point with the desired motion, and that the swing looks natural as it starts from 0, ends from 0, and hits the desired velocity in the middle (at ball contact).

Since the robot has more degrees of freedom than strictly needed to achieve the primary task, there are different joint configurations that can place the paddle at the same location. This redundancy allows for the possibility to incorporate a secondary task. In this project, a secondary task was implemented to keep the added prismatic joints at the paddle close to zero. This biases the robot toward hitting the ball near the center of the paddle rather than near its edges. This secondary task was projected into the nullspace of the Jacobian so that it would not disrupt the primary task of placing the paddle correctly. The following equations were used during implementation:

$$\dot{\mathbf{q}} = J^{\dagger}\,\mathbf{e} + \left(I - J^{\dagger}J\right)\,\dot{\mathbf{q}}_{\text{sec}}$$

Since there are multiple ways to arrive at the same point in space with this arm, we wanted to select the positions that look most human-like and natural. The inverse kinematics problem has multiple valid solutions, but we want to bias the solver towards specific branches.

The first way of doing this is to decide whether to hit backhand or forehand. We decide which side of the paddle to hit with based on the relative position of the paddle position and the goal. This tells us whether we hit with the normal vector of the paddle (forehand) to negative normal vector (backhand). This ensures the paddle faces the ball appropriately at contact, and doesn't twist itself unnecessarily if the other side of the paddle is already facing the ball.

Additionally, in our Newton-Raphson, we attempt to "smartly" provide an initial guess for q based on the position of the ball and the elbow up vs elbow down solution. This helps us essentially choose which branch the Newton-Raphson will follow. We therefore provide an informed guess for the joint angles based on the ball's position and expected elbow configuration, by biasing the yaw joint towards the angle from the ball to the plane of the arm $\theta = atan2(y, x)$, we then bias the elbow joint towards one of two postures $(+/-\pi/2$ based on the target's quadrant (intercept lies in front of robot vs behind), in both cases using a blending factor (weight) so the solver remains stable. We then re-center the prismatic joints on the initial guess. This improves convergence reliability and produces more natural motions.

We also attempted to limit the motion of the elbow joint (joint4) such that it does not go past 180 degress, just like humans. However after testing we found that this created

unnatural movement in the other joints upon attempting to converge, so we chose to go with the forehand/backhand and inital guess methods to create natural movement.

Overall, three implementation details were particularly important to achieve the desired motion:

- **Damping and weighting for robustness:** Damping the inverse reduces the sensitivity near singularities and improves the stability of the robot. Weighting helps avoid the motion of undesired joints and helps simulate a more natural motion.
- **Spline-based execution:** Instead of integrating joint velocities for the entire motion, the forward spline was computed using inverse kinematics and after interception, the spline back allowed for the system to reset between iterations.
- **Natural Movement:** Allowing the arm to converge to a forehand or backhand solution, and providing an informed initial guess to the Newton-Raphson proved to be the two methods that got us the most realistic and reliable motion under varied conditions.

Singularities arise when the Jacobian matrix becomes ill-conditioned and this causes small task-space motions to require very large joint velocities. Under this implementation, singularities are handled implicitly through the use of the damped pseudoinverse. This helps improve numerical stability and ensures that the inverse kinematics solver continues to produce reasonable joint motions even when the arm approaches limits. Additionally, weighting certain joints further reduces the likelihood of entering unstable configurations by discouraging motions in joints that are prone to causing singularities.

Particular Features and Cases Handled

Several features were implemented into the project. One of such features was the addition of the two prismatic joints at the paddle, and as previously described, it increases the effective workspace (Figure 1). This project also handles collisions between the ball and paddle. The paddle is represented as a rectangular surface, and a collision is detected when the ball lies within the paddle's projected area in the tangential direction and is within one ball radius of the paddle surface in the normal direction. This ensures physical contact with the paddle. Additionally, collisions are only processed when the ball is moving towards the paddle, preventing interactions when the ball is separating from the surface. These safeguards improve the reliability of collision detection.

The biggest issue was still folding in on itself (thus twisting unnaturally) when trying to hit a ball near its base. For example, when you're playing tennis, you don't wait till the ball is at your stomach before hitting it. When the ball is above you, you try to hit it above you. When the ball has fallen to the side, you might want to hit it lower (forehand or backhand) for more natural movements. This is why we constrained the z-hit coordinate to be within a hemisphere, so the arm could adjust where it wanted to intercept the ball based on the ball's initial location, which helped tremendously with the unnatural twisting. After tuning parameters, we decided that having the arm hit the ball when it is 0.75 away would be the best, so we defined a sphere of radius 0.75m for the arm to intercept the ball. Wherever the ball fell, we would calculate when it could hit a "shell" of radius 0.75m around the arm, and thus calculating the z-hit coordinate for each ball uniquely (x,y predetermined when point is chosen for ball to randomly fall

from). This would look like hitting it up high, or reaching outwards to hit it forehand or backhand. But either way, it wouldn't wait for the ball to fall very low and try to twist itself to catch it. When the ball falls outside of this 0.75m-radius shell, that means the arm has to reach out anyways to try to hit the ball. We default the z-hit height (if there's no solution on the 0.75m shell) to 0.15. This means that the arm would reach out and try to hit it pretty low, and is guaranteed to not twist upon itself to try to reach it. If it is out of the range of motion of the arm, the damped inverse handles this singularity and moves the paddle as close as possible to try to hit it. If the ball falls within the task space of the arm, and is outside the shell, the robot would have to reach out to hit it anyways, and the height would be set to 0.15m.

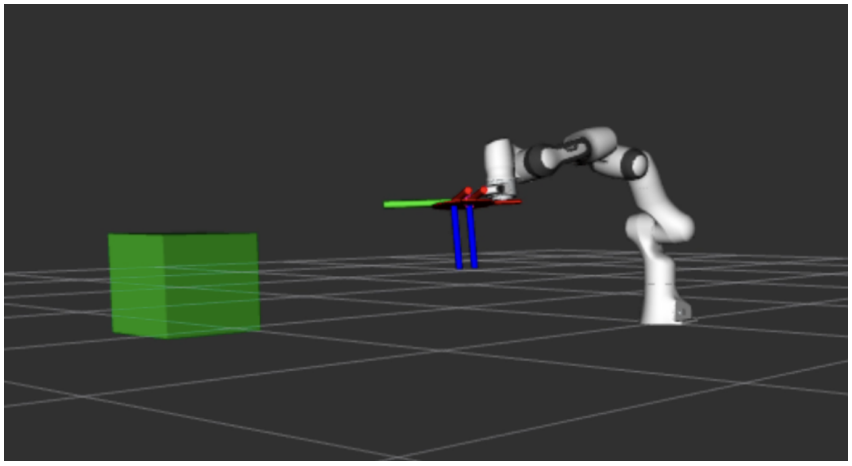


Figure 1: Tree Frames highlighting the two prismatic joints on the paddle such that the ball can hit anywhere within that area

Because the points at which the paddle intercepts the ball are constrained to occur on the perimeter (surface) of a spherical region centered at the robot base, this limits planning to reachable configurations and avoids commanding the robot to intercept the ball in regions that feel less natural, such as when the balls comes too close to the robot and it would feel awkward even for a human arm to hit it at that configuration.

The implementation also supports forehand and backhand interactions. Note that even though we spawn the ball in reachable locations we conducted extensive testing on how that system reacts if the arm is unable to reach the ball to handle singularities.

Singularities are also taken into consideration at the planning level. By constraining intercept points to lie on the surface of a sphere centered at the robot base, the system avoids planning motions near the boundary of the robot's workspace.

Analysis

In addition to implementing the core framework, we also explored several design choices that focused on how different strategies improve how natural the robot moves. One approach taken was focusing on the effect of initial guesses in the inverse kinematics solver. This way the solver would locally converge, meaning that better initial guesses would cause a faster convergence and more reasonable joint configurations. This approach was considered because oftentimes, the robot needed to hit the ball from awkward angles such as when the arm was in between the ball and the arm. Ideally, this approach would modify the initial guess based on where the ball spawned. After this approach didn't fully address the issue, we tried a different method. We then constrained the intercept points to lie on the perimeter of a sphere centered at the robot base, limiting the task to well-defined reachable regions on the workplace that significantly made the robot motion more natural.

We also tuned parameters to balance stability and physical realism. In the inverse kinematics solver, the damping coefficient and joint-space weighing were adjusted to prevent excessive joint motion near singularities while maintaining accurate task execution. Increasing damping improved numerical stability but reduced responsiveness, and lower damping led to faster convergence with a few observed oscillating solutions. We ultimately decided on, for the weighted inverse, using

```
w = np.array([1.0, 1.0, 5.0, 10.0, 5.0, 1.0, 1.0, 1.0, 1.0])
W = np.diag(w)
```

to constrain the joints 3,4,5 because those were the elbow, forearm, and bicep rotation. The most obvious unnatural positions were when these joints were overly rotated, and this damping helped constrain the joint movements a bit.

We decided that the secondary task for the elbow (keeping it near 90 degrees) was more helpful than the weighted inverse, through observing the arm's motion when balls fell close to it and it would intercept them low, the secondary task helped prevent the arm from wrapping in on itself (elbow value too small).

However, because the primary task is strict, the solver will still mvoe expensive joints when necessary, so weighting alone didn't eliminate extreme configurations. To address folding inward when intercepts occur closer to the robot, we added an explicit secondary task in the Jacobian nullspace that biases selected joints towards a comfortable posture

and recenters the prismatic sliders. The secondary term does not compete with the primary 6D pose objective, but it affects the redundant solution branch the solver converges to.

We also tuned solver and stability parameters empirically. The damping was increased when solutions became unstable near singular configurations, step size c was reduced when the Newton iterations overshot or always oscillated solutions (c=5 was wayyy too big, we found that c=0.2 is best) The stopping tolerances tol and dq_{tol} were chosen to distinguish true convergence from 'stuck' cases. These tolerances were tweaked to around 1e-3, as they were originally 1e-5, which was too small and would have trouble converging. We also selected a fixed iteration task for Newton Raphson to be 500. We also introduced a gain of 0.5 on the secondary task so the posture shaping was noticeable without overwehlming the primary task. We used a coefficient of restitution of 1 because when we were trying to determine the correct equation for collision and momentum calculations, (while debugging) we didn't want the coefficient of restitution to be the reason that the ball would miss the target (because the calculated velocity wouldn't match the outgoing ball velocity because the ball would lose energy outgoing). But with teh finalized physics equation, the coefficient of restitution can be tweaked, and the ball works.

appendix

$$m_b v_b^- + m_p v_p^- = m_b v_b^+ + m_p v_p^+$$

$$v_b^+ - v_p^+ = -e(v_b^- - v_p^-)$$

$$= -e v_b^- + e v_p^-$$

$$v_p^+ = v_b^+ + e v_b^- - e v_p^-$$

$$m_b v_b^- + m_p v_p^- = m_b v_b^+ + m_p(v_b^+ + e v_b^- - e v_p^-)$$

$$m_b v_b^- + m_p v_p^- = m_b v_b^+ + m_p v_b^+ + e m_p v_b^- - e m_p v_p^-$$

$$m_b v_b^- + m_p v_p = (m_b + m_p) v_b^+ + e m_p v_b^- - e m_p v_p^-$$

$$(m_b - e m_p) v_b^- + (m_p + e m_p) v_p^- = (m_b + m_p) v_b^+$$

$$v_b^+ = \frac{(m_b - e m_p) v_b^- + (1+e) m_p v_p^-}{m_b + m_p}$$

Figure 2: Collision derivation (from momentum)

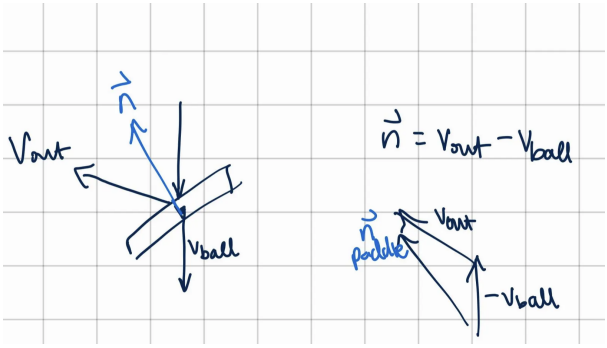


Figure 3: Calc paddle normal vector